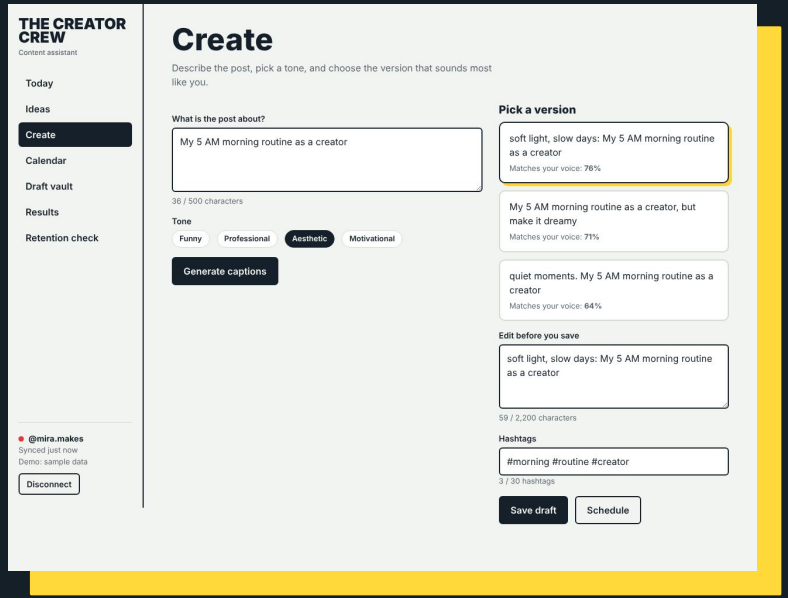
SOFTWARE ENGINEERING MODEL
COURSE PROJECT

The Creator Crew

An AI content assistant for social media creators, and the project where I learned FastAPI, SQLAlchemy and Selenium.



Mahek Shukla

Five tools to post once



Write

Hours lost to captions and hashtags



Store

Drafts scattered across apps



Time

Posting times are guesses



Approve

Feedback buried in DMs



Learn

Drop-off points stay invisible

One workspace replaces all five.

One workspace, six jobs

Create

Three captions and hashtags from one prompt

FR-1 to FR-3

Manage

Searchable drafts, rollback to any of 5 versions

FR-4, FR-5

Personalize

Ranks suggestions against your past posts

FR-6

Distribute

Suggests a time, adapts format per platform

FR-7 to FR-9

Optimize

Predicts drop-off and suggests stronger hooks

FR-10, FR-12

Collaborate

Reviewers approve or request rework, fully logged

FR-11

Captions that sound like you

1

Pick a tone

Funny, Professional, Aesthetic or Motivational shapes every variant.

2

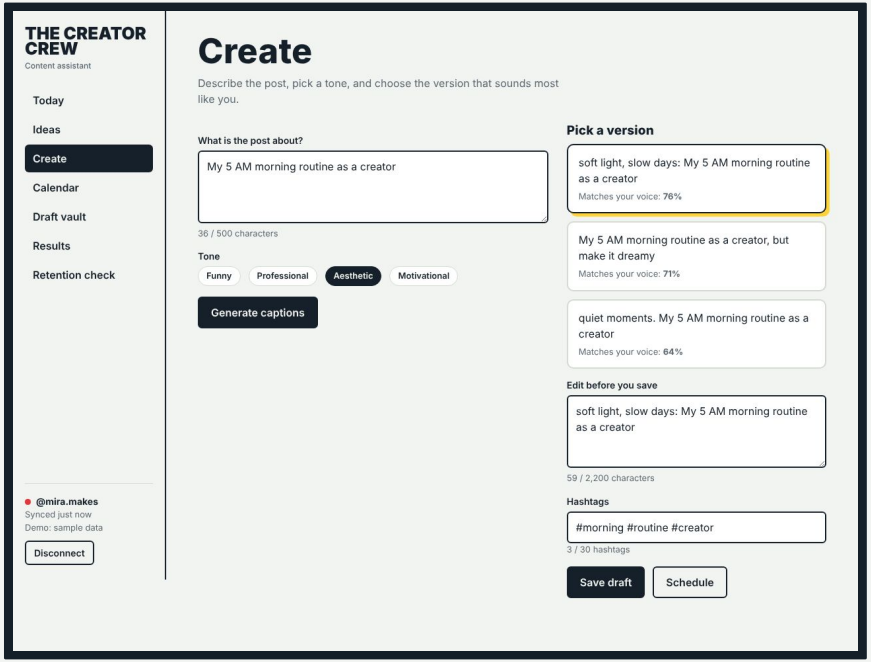
Compare three versions

Each shows how closely it matches your voice, learned from your past posts.

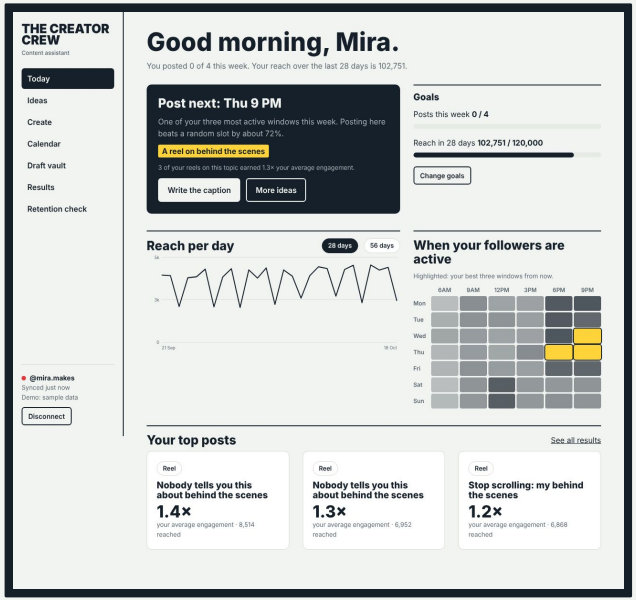
3

Edit, then save or schedule

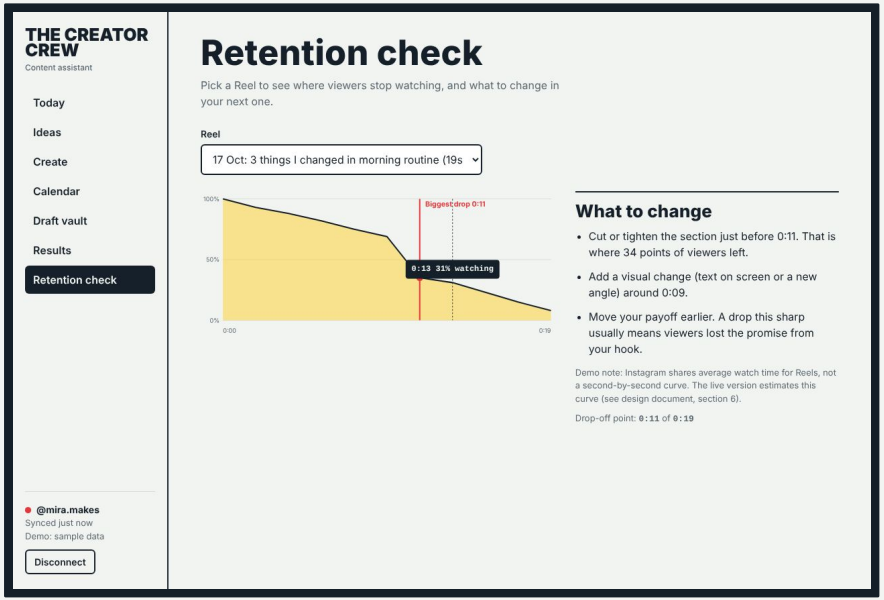
Hashtags, character counts and the next step sit on one screen.



When to post. Where they leave.

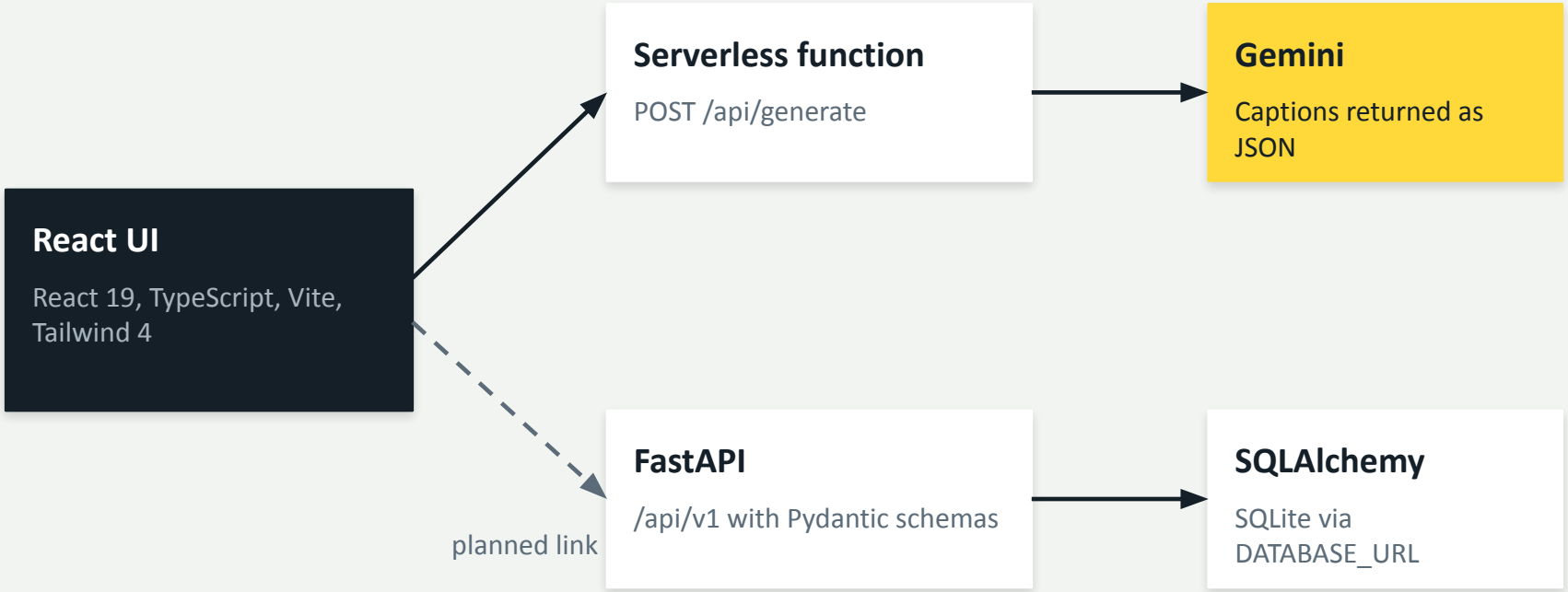


Today: your three best posting windows, from a follower-activity heatmap (sample data)



Retention check: biggest drop at 0:11, with concrete edits for the next reel (sample data)

How it fits together



Solid: running today. Dashed: next step.

FastAPI: contract first

```
@app.post("/api/v1/generate",
          response_model=GenerateResponse)
def generate_captions(
        req: GenerateCaptionRequest):
    return ai_adapter.generate(
        req.prompt, req.tone)
```

backend/app/main.py

11 Pydantic schemas guard every request and response

3 s caption target; latency returns with each call

WHAT IT TAUGHT ME

Schemas are the spec

Bad input is rejected before my code runs. Docs generate themselves.

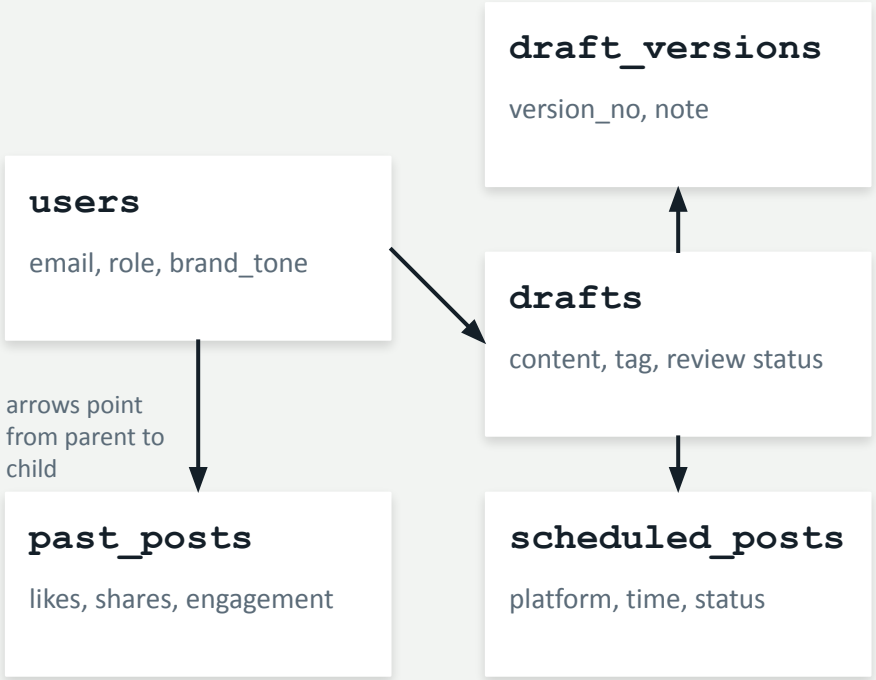
Hide the provider

A swappable AI interface lets a mock stand in for Gemini.

Config lives in the environment

Database URL and JWT secret come from env vars, not code.

SQLAlchemy: modelling history



WHAT IT TAUGHT ME

Relationships do the joins

back_populates keeps user.drafts and draft.user in sync.

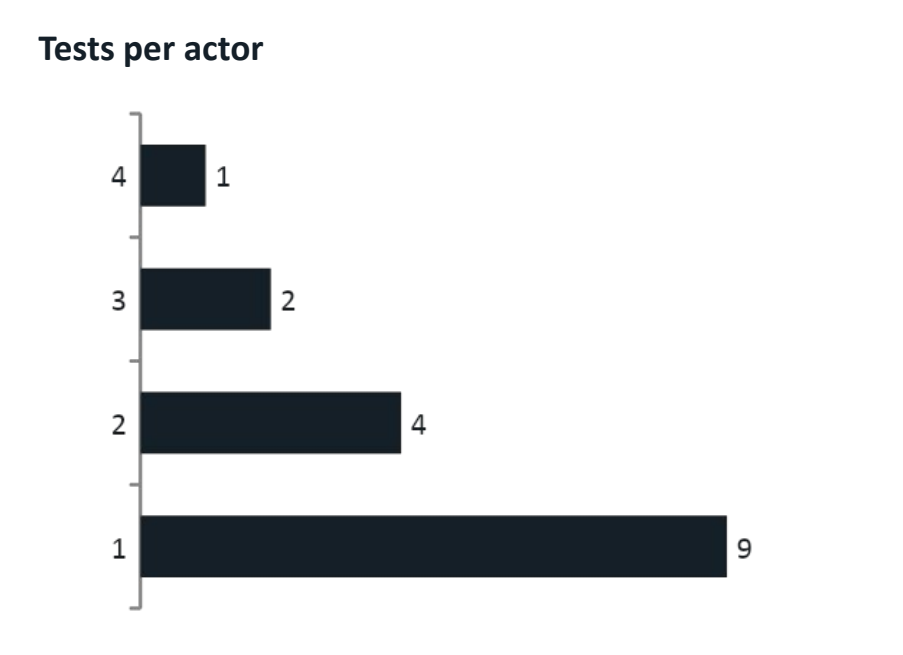
History is a table

Every revision is a row. Delete a draft and its history goes too.

One session per request

A get_db dependency opens a session and always closes it.

Selenium: 16 tests, 3 actors



WHAT IT TAUGHT ME

Start from the use cases

Each test maps to one actor and one use case, so coverage reads like the spec.

Wait, then assert on state

Explicit waits, then checks on aria-pressed, toasts and audit-log rows.

Cover the undo paths

Delete with undo, rework requests and version rollback each get a test.

Tests that run on every push

1

Push

Any push or pull request to main

2

Build

npm run build, then serve on port 3000

3

Wait

wait-on holds until the app answers

4

Test

Headless Chrome runs all 16 tests with pytest

5

Report

report.html uploads as an artifact for 14 days

Two modes, one suite

Headed with slow motion for live demos. Headless in CI. One flag switches them.

What’s real, what’s next

Working today

- React UI: Today, Ideas, Create, Calendar, Vault, Results, Retention, Team
- Gemini captions through a serverless function
- FastAPI service, typed schemas, five SQLAlchemy models
- 16 Selenium tests and a CI workflow

Next

- Build the draft, auth and scheduling routes (drafts is a stub today)
- Swap the mock adapter for Gemini
- Add Alembic migrations (pinned, no revisions yet)
- Point Selenium at the live API, not sample data

Three tools, one project

FastAPI

Typed endpoints and self-writing docs

SQLAlchemy

Relational models with versioned history

Selenium

16 use-case tests, run in CI

See the code

github.com/Mahek-Technophile/The_Creator_Crew